

Name : Kanika Saini

Program: Btech CSE B16

SAP ID: 590034098

HDOC Day-25

Problem Statement

Write a menu-driven C program to display:

1. Right-aligned increasing triangle using *
2. Right-aligned increasing number triangle
3. Right-aligned decreasing triangle using *
4. Exit

The program uses a **switch statement**, **nested loops**, **no arrays**, and a separate parameterless `void` function for each pattern.

1. Algorithm

Main Program Algorithm

Step 1: Start.

Step 2: Declare an integer variable `choice`.

Step 3: Display the menu:

1. Right-aligned increasing triangle
2. Right-aligned increasing number triangle
3. Right-aligned decreasing triangle
4. Exit

Step 4: Input the user's choice.

Step 5: Use a `switch` statement.

Step 6: If `choice = 1`, call `increasingTriangle()`.

Step 7: If `choice = 2`, call `numberTriangle()`.

Step 8: If `choice = 3`, call `decreasingTriangle()`.

Step 9: If `choice = 4`, terminate the program.

Step 10: Otherwise, display "Invalid choice!".

Step 11: Repeat the menu until the user selects Exit.

Step 12: Stop.

2. Algorithm for Right-Aligned Increasing Triangle

The sheet asks for an increasing right-aligned asterisk triangle.

Step 1: Start the function.

Step 2: Input the number of rows `n`.

Step 3: Set `i = 1`.

Step 4: Repeat for every row until `i <= n`.

Step 5: Print `n-i` groups of spaces.

Step 6: Print `i` asterisks.

Step 7: Move to the next line.

Step 8: Repeat until all rows are printed.

Step 9: Return to the main program.

Example for `n = 4`

*

```
  * *
 * * *
* * * *
```

3. Algorithm for Right-Aligned Increasing Number Triangle

The PDF specifies that each row should contain natural numbers from 1 up to that row number.

Step 1: Start the function.

Step 2: Input number of rows n .

Step 3: Set $i = 1$.

Step 4: Repeat for every row until $i \leq n$.

Step 5: Print the required leading spaces.

Step 6: Using an inner loop, print numbers from 1 to i .

Step 7: Move to the next line.

Step 8: Continue until n rows are printed.

Step 9: Return to main program.

Example for $n = 4$

```
      1
     1 2
    1 2 3
   1 2 3 4
```

4. Algorithm for Right-Aligned Decreasing Triangle

The third option is a decreasing right-aligned asterisk triangle.

Step 1: Start the function.

Step 2: Input number of rows n .

Step 3: Start i from n .

Step 4: Print the required leading spaces.

Step 5: Print i asterisks using an inner loop.

Step 6: Move to the next line.

Step 7: Decrease the number of asterisks by one.

Step 8: Repeat until one asterisk remains.

Step 9: Return to main program.

Example for $n = 4$

```
* * * *
 * * *
  * *
   *
```

5. Test-Case Table

Test Case	Menu Choice	Input n	Expected Result	Actual Result	Status
1	1	4	Right-aligned increasing * triangle with 4 rows	Same as expected	Pass
2	2	4	Right-aligned number triangle from 1 to row number	Same as expected	Pass
3	3	4	Right-aligned decreasing * triangle with 4 rows	Same as expected	Pass
4	1	3	Increasing triangle with 3 rows	Same as expected	Pass
5	4	—	Program exits	Program terminated	Pass
6	7	—	Invalid choice!	Invalid choice!	Pass

6. Complete C Program

```

#include <stdio.h>

void increasingTriangle();
void numberTriangle();
void decreasingTriangle();

int main()
{
    int choice;

    do
    {
        printf("\n===== PATTERN MENU =====\n");
        printf("1. Right-aligned increasing triangle\n");
        printf("2. Right-aligned increasing number triangle\n");
        printf("3. Right-aligned decreasing triangle\n");
        printf("4. Exit\n");

        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch(choice)
        {
            case 1:
                increasingTriangle();
                break;

            case 2:
                numberTriangle();
                break;

            case 3:
                decreasingTriangle();
                break;

            case 4:
                printf("Exiting the program...\n");
                break;

            default:
                printf("Invalid choice! Please enter 1 to 4.\n");
        }

    } while(choice != 4);

    return 0;
}

void increasingTriangle()
{
    int n, i, j;

    printf("Enter number of rows: ");
    scanf("%d", &n);

    for(i = 1; i <= n; i++)

```

```

    {
        for(j = 1; j <= n - i; j++)
        {
            printf("  ");
        }

        for(j = 1; j <= i; j++)
        {
            printf("* ");
        }

        printf("\n");
    }
}

```

```

void numberTriangle()
{
    int n, i, j;

    printf("Enter number of rows: ");
    scanf("%d", &n);

    for(i = 1; i <= n; i++)
    {
        for(j = 1; j <= n - i; j++)
        {
            printf("  ");
        }

        for(j = 1; j <= i; j++)
        {
            printf("%d ", j);
        }

        printf("\n");
    }
}

```

```

void decreasingTriangle()
{
    int n, i, j;

    printf("Enter number of rows: ");
    scanf("%d", &n);

    for(i = n; i >= 1; i--)
    {
        for(j = 1; j <= n - i; j++)
        {
            printf("  ");
        }

        for(j = 1; j <= i; j++)
        {
            printf("* ");
        }
    }
}

```

```
    }  
    printf("\n");  
}  
}
```

7. Program Execution

Test 1 — Increasing Triangle

```
(kali㉿kali)-[~/Desktop]  
$ cc day25.c  
(kali㉿kali)-[~/Desktop]  
$ ./a.out  
===== PATTERN MENU =====  
1. Right-aligned increasing triangle  
2. Right-aligned increasing number triangle  
3. Right-aligned decreasing triangle  
4. Exit  
Enter your choice: 1  
Enter number of rows: 4  
  *  
 * *  
* * *  
* * * *
```

Test 2 — Increasing Number Triangle

```
===== PATTERN MENU =====  
1. Right-aligned increasing triangle  
2. Right-aligned increasing number triangle  
3. Right-aligned decreasing triangle  
4. Exit  
Enter your choice: 2  
Enter number of rows: 4  
  1  
 1 2  
1 2 3  
1 2 3 4
```

Test 3 — Decreasing Triangle

```
==== PATTERN MENU ====
1. Right-aligned increasing triangle
2. Right-aligned increasing number triangle
3. Right-aligned decreasing triangle
4. Exit
Enter your choice: 3
Enter number of rows: 4
* * *
 * *
  *
   *
```

Test 4 — Invalid Choice

```
==== PATTERN MENU ====
1. Right-aligned increasing triangle
2. Right-aligned increasing number triangle
3. Right-aligned decreasing triangle
4. Exit
Enter your choice: 7
Invalid choice! Please enter 1 to 4.
```

Test 5 — Exit

```
==== PATTERN MENU ====
1. Right-aligned increasing triangle
2. Right-aligned increasing number triangle
3. Right-aligned decreasing triangle
4. Exit
Enter your choice: 4
Exiting the program...
```

Conclusion

The program successfully implements all **three right-aligned patterns** specified in the HDOC Day-25 activity using **nested loops, switch-case, and separate parameterless void functions without arrays**, matching the requirements given in the assignment.